MakeYourMove

Platform Assessment — Implementation Specification

Document version	2.0 (Revised architecture)
Target audience	CS students (Year 1–4), early-career engineers
Domains covered	7 (all domains fully specified)

Behavior-driven career discovery for software engineering students

1. System Overview

MakeYourMove is a behavior-driven career exploration platform. Unlike personality quizzes or preference surveys, it observes how users think through engineering-style problems and uses those behavioral signals — not self-reported preferences — to surface the software domains most likely to suit them.

1.1 Core Principles

- Behavior over self-reporting: every recommendation is earned through interaction, not by asking 'what do you prefer?'
- Transparency over black box: users see why they were matched, with specific reference to what they did during the session.
- Exploration over labeling: results are presented as an invitation to investigate, not a verdict.
- No external tool installs: all validation tasks run entirely in-browser. No Weka downloads, no Postman setup.

1.2 Assessment Flow at a Glance

#	Stage	Purpose
1	**Context intake**	3 quick questions to personalise activity order and weighting
2	**Behavioural screening (Pre-Assessment)**	6 interactive micro-tasks covering all 7 domains — captures thinking instincts
3	**Thinking style reveal**	3–4 named styles surfaced with AI-generated, session-specific explanations
4	**In-platform validation tasks**	2 live tasks per shortlisted style — no downloads, fully in-browser
5	**Confidence score update**	Rule-based engine recalculates domain weights from validation behavior
6	**Domain bubble exploration**	Weighted bubbles open to roadmaps, tasters, and curated resources
7	**Personalised learning roadmap**	AI reorders and annotates each roadmap based on demonstrated skill gaps

2. Stage 1 — Context Intake

Before any activity begins, the platform collects three lightweight signals. This takes under 30 seconds and is used only to personalise the order and initial weighting of pre-assessment activities — not to make any domain recommendation.

2.1 The Three Questions

Question	Options	Effect on assessment
What year of study are you in?	*Year 1 / Year 2 / Year 3 / Year 4+ / Working professional*	Year 1–2 users see Activity 1 (system flow) first; Year 3–4 see Activity 4 (architecture) first
Have you tried any domain hands-on before?	*Never tried any / Dabbled in one or two / Built something in one domain*	Prior experience raises the threshold before that domain is shortlisted — prevents confirmation bias
What are you hoping to figure out today?	*No idea where to start / Narrowing between 2–3 options / Validating a choice I've already made*	Sets tone: exploratory users get wider bubble spread; validators get tighter depth-first routing

3. Stage 2 — Pre-Assessment (Behavioural Screening)

Six interactive micro-tasks. Each takes 20–40 seconds. Total time under 4 minutes. All tasks are behavioral — there are no multiple-choice questions about preferences. The system records what the user does, in what order, and how quickly — not what they say they would do.

Domain coverage: Activities 1–4 cover Systems/Backend/DevOps/Frontend/AI. Activities 5 and 6 specifically cover Cybersecurity and Product Engineering, which were absent from the original design.

Activity 1 — System Flow Arrangement

Field	Detail
Domain signal	Backend, DevOps, AI/ML, Frontend
Duration	25–35 seconds
Interaction type	Drag and drop — sequence ordering
Objective	Detect whether the user thinks in terms of system architecture, data flow, or UI-first

Task Description

The user sees 6 unlabelled component blocks in a randomised pile:

- User Request
- API Gateway
- Application Server
- Database
- ML Model
- UI Response

They drag these into a horizontal sequence representing a logical application flow. There is no single correct answer — the arrangement itself is the data.

Behavioral Signals

Signal	Behavior	Domain indicated
API + DB prioritised early	User builds the data path before UI concerns	Backend
ML Model included in flow	User finds a natural place for the model between data and response	AI/ML
UI Response placed first or last with deliberate thought	User anchors flow around user experience endpoints	Frontend

Load balancer or monitoring mentioned in retry	User revisits arrangement, adds infrastructure thinking	DevOps
Multiple rearrangement attempts	User is exploring — broader curiosity signal, lower certainty for any one domain	Hybrid

Activity 2 — Messy UI Fix

Field	Detail
Domain signal	Frontend, Product Engineering
Duration	25–40 seconds
Interaction type	Drag and drop — spatial repositioning
Objective	Detect UX intuition, visual hierarchy awareness, and product thinking

Task Description

The user sees a poorly designed registration form with seven elements scrambled in layout:

- Page title (misaligned, tiny font)
- Email input field (at the bottom)
- Password input field (above the email field)
- Confirm password field (far right, orphaned)
- Error message (floating unattached in the middle)
- Terms checkbox (above the headline)
- Submit button (top-left, above all inputs)

The user drags elements to positions they feel make more sense. A faint grid overlay helps with alignment. No instructions are given about what 'correct' looks like.

Behavioral Signals

Signal	Behavior	Domain indicated
Error message placed adjacent to its triggering field	User pairs error state with the input it belongs to	Frontend / Product
Title moved to top, button to bottom	User enforces reading-order hierarchy	Frontend
Confirm password placed directly below password	User groups related inputs — field-level UX thinking	Product Engineering
Terms checkbox moved to near the button	User applies consent-placement logic from real-world patterns	Product Engineering
User spends time on the error message placement specifically	Attention to failure states — a quality/reliability instinct	Backend / DevOps

Post-Task Reaction Card

After submitting, a single reaction question appears — not a quiz:

> **What drove your biggest change?**
> A — Something felt visually off when I looked at it as a whole
> B — I thought about the order someone would fill this out step by step
> C — I wasn't sure — I tried a few positions and picked the one that felt least wrong

All three answers are valid. The signal is in the reasoning mode, not the correctness.

Activity 3 — Live Debug Dashboard

Field	Detail
Domain signal	Backend, DevOps, Frontend
Duration	30–45 seconds
Interaction type	Click-to-pin investigation — spatial behavior, not menu selection
Objective	Detect debugging instinct and system investigation strategy

Task Description

A simplified application dashboard is shown. All panels are visible simultaneously — the user is not presented with a menu. The panels shown are:

- DB query log — slow queries highlighted
- Network tab — showing 3 API calls with response times
- Server CPU chart — a spike visible at 14:32
- Frontend render timeline — normal
- Deployment history — a release at 14:28

The user clicks a 'Start here' pin and drags it onto whichever panel they would investigate first. They can then place a 'Check next' pin on their second choice. The interaction is a spatial drag, not a dropdown.

Behavioral Signals

Signal	Behavior	Domain indicated
Pin placed on DB query log first	Database-first investigation instinct	Backend
Pin placed on Deployment history first	Infrastructure and release causality instinct	DevOps
Pin placed on Server CPU chart first	System performance and resource-level thinking	DevOps / Backend

Pin placed on Network tab first	API and service boundary thinking	Backend
Pin placed on Frontend render first	Client-side performance awareness	Frontend
User hovers over deployment history after placing first pin elsewhere	Secondary awareness of infrastructure — partial DevOps signal	DevOps (secondary)

Activity 4 — Architecture Builder

Field	Detail
Domain signal	DevOps, Backend, AI/ML, Cybersecurity
Duration	35–45 seconds
Interaction type	Drag into build zone — component selection and placement
Objective	Evaluate architecture thinking, infrastructure awareness, and security instinct

Task Description

Scenario presented: 'This product just hit 1 million users. Build the core infrastructure.' The user sees 9 component tiles in a panel and drags any they deem necessary into a build zone:

- Load balancer
- Application server
- Primary database
- Redis cache
- Monitoring dashboard
- Docker containers
- ML recommendation model
- Firewall
- CDN

There is no maximum — users can add all 9 or just 2. The selection and order of additions are both recorded.

Behavioral Signals

Signal	Behavior	Domain indicated
Load balancer + monitoring added early	Infrastructure reliability first	DevOps
Redis + database prioritised	Data layer and caching awareness	Backend

ML model included	Intelligence layer thinking — curious about AI features at scale	AI/ML
Firewall added (especially early)	Security-first instinct — often missed by non-security thinkers	Cybersecurity
CDN included	Content delivery and latency awareness	Frontend / DevOps
User adds all 9 components	Comprehensive coverage instinct — possible DevOps or Product signal	DevOps / Product
User adds only 2–3 focused components	Prioritisation and constraint-awareness — strong signal quality	Domain of chosen 2–3

Activity 5 — Threat Spotter (NEW)

Field	Detail
Domain signal	Cybersecurity (primary), Backend (secondary)
Duration	30–40 seconds
Interaction type	Drag flag marker onto identified vulnerabilities
Objective	Detect adversarial thinking, security instinct, and trust-by-default vs trust-by-verification mindset

Task Description

The user sees a login page screenshot — functional and visually normal. Three vulnerabilities are embedded at different levels of obviousness:

1. Obvious: Password visible in plain text in the DOM (shown via a browser dev-tools panel overlay at the side)
2. Medium: No rate-limit indicator — the submit button shows no throttle or CAPTCHA
3. Subtle: The session token is exposed in the URL bar after login (?token=eyJhbGc...)

The user is given three red flag markers and asked to drag them onto anything they think looks wrong. They do not need to use all three.

Behavioral Signals

Signal	Behavior	Domain indicated
Flags URL token before the DOM password	URL and transport-layer attention — strong Cybersecurity signal	Cybersecurity (high)
Flags all three correctly	Comprehensive threat surface awareness	Cybersecurity (high)
Flags DOM password only	Notices the obvious vulnerability — basic awareness, partial signal	Cybersecurity (low)
Flags the missing CAPTCHA	Rate-limiting and abuse-prevention awareness — backend security thinking	Backend / Cybersecurity
Places no flags or only 1 after long hesitation	Domain is unfamiliar — negative Cybersecurity signal, valid result	Not Cybersecurity
Speed to first flag under 8 seconds	Threat scanning is an instinct, not a deliberate search	Cybersecurity (strong)

Activity 6 — Feature Triage (NEW)

Field	Detail
Domain signal	Product Engineering (primary), AI/ML (secondary)
Duration	35–45 seconds
Interaction type	Drag cards into three buckets — ship, reconsider, cut
Objective	Detect product judgment, prioritisation instinct, and user-versus-technical tradeoff reasoning

Task Description

Eight feature request cards are shown in a backlog column. Each card has a one-line title and a two-line description. The three destination buckets are:

- Ship now — high confidence, clear user value
- Needs more thought — promising but something's unresolved
- Cut it — not worth building right now

The feature cards are:

Feature	Description	Intended tension
Dark mode	Top-requested by users. Low engineering complexity. No new data needed.	*Easy yes — tests if user overcomplicates it*
AI-generated onboarding tour	Personalised first-run guide using user data. High complexity, unproven uplift.	*Complexity vs. delight tradeoff*
Export to CSV	Power users want raw data access. 3 complaints this week, 50k happy users.	*Loud minority vs. silent majority*
In-app notifications	Engagement team wants push alerts. No user research done yet.	*Business pressure vs. missing evidence*
Profile badges	Gamification layer. Fun. Low complexity. No evidence it retains users.	*Vanity feature test*
Two-factor authentication	Security improvement. No user asked for it. Compliance may require it soon.	*Need vs. demand gap*
Offline mode	Requested twice. Would triple the mobile engineering effort.	*Effort vs. demand mismatch*
Waitlist for premium tier	Revenue idea. Requires pricing strategy to be defined first.	*Dependency and readiness test*

Behavioral Signals

Signal	Behavior	Domain indicated
2FA moved to Ship — even unprompted by users	Compliance and security thinking over demand	Cybersecurity / Product

In-app notifications moved to Needs more thought	Evidence-based decision — won't ship without user research	Product Engineering (strong)
AI onboarding moved to Ship immediately	Appetite for ML features regardless of complexity	AI/ML
Profile badges cut confidently	Vanity feature recognition — impact-over-engagement instinct	Product Engineering
Most cards go to Needs more thought	Cautious, evidence-first thinker — strong product signal	Product Engineering
Cards decided quickly with few hesitations	Confident prioritisation instinct, not paralysed by ambiguity	Product Engineering (decisive type)

4. Stage 3 — Thinking Style Reveal

After the 6 activities, the rule-based scoring engine aggregates domain signals and selects 3–4 thinking styles to surface. The user does not see a score or a percentage. They see named style cards — each with an AI-generated explanation drawn from their specific session behavior.

Selection logic: Styles are shortlisted by relative score gap, not absolute threshold. If two styles are within 5% of each other and the 4th is clearly weaker, only 3 cards are shown. If four are clustered closely, all four appear.

AI role: The style selection is fully rule-based and auditable. AI only writes the personalised explanation copy beneath each card — referencing specific actions the user took during the session.

4.1 The Eight Thinking Styles

People Instinct — drawn to humans, behaviour, experience

The Experience Shaper

"How will this feel to someone using it?"

Behavioral signal: Moves error messages next to their triggering fields. Builds UI flow from the user's reading order, not the engineer's convenience.

Maps to: Frontend Development, Product Engineering

The Problem Definer

"Before we build anything — what problem are we actually solving?"

Behavioral signal: Moves 'In-app notifications' to Needs More Thought because no user research exists. Cuts profile badges without hesitation.

Maps to: Product Engineering, AI/ML

Systems Instinct — drawn to structure, reliability, scale

The Systems Thinker

"How do the pieces connect and what breaks first under load?"

Behavioral signal: Builds API → DB → cache before touching UI. Adds monitoring and load balancer before the application server itself.

Maps to: Backend Engineering, DevOps / Infrastructure

The Reliability Keeper

"Is this stable? What happens when it fails at 3am?"

Behavioral signal: Places the Start Here pin on the deployment history immediately. Correlates the CPU spike with the 14:28 release without prompting.

Maps to: DevOps / Infrastructure, Backend Engineering

Evidence Instinct — drawn to signals, patterns, unknowns

The Signal Seeker

"There's a pattern in this noise — I want to find it."

Behavioral signal: Includes the ML model in the system flow between the database and the UI response. Investigates the DB query log before the CPU chart.

Maps to: Data Engineering, AI/ML

The Hypothesis Tester

"Let me try something, measure it, and see what breaks the assumption."

Behavioral signal: Rearranges the system flow multiple times before settling. Moves the AI onboarding feature to Ship Now despite the complexity flag.

Maps to: AI/ML, Data Engineering

Defence Instinct — drawn to adversarial thinking, trust, risk

The Threat Anticipator

"I'm already thinking about how this gets exploited."

Behavioral signal: Flagged the session token in the URL within 8 seconds — before noticing the visible DOM password. Added the firewall first in the architecture builder.

Maps to: Cybersecurity, Backend Engineering

The Trust Architect

"Who should have access to what, and how do we verify it?"

Behavioral signal: Moved 2FA to Ship Now without being asked, citing compliance. Flagged the missing CAPTCHA as a rate-limiting concern.

Maps to: Cybersecurity, DevOps / Infrastructure

5. Stage 4 — In-Platform Validation Tasks

For each shortlisted thinking style, the user is presented with two validation tasks that run entirely in-browser. These tasks are designed to confirm genuine engagement with domain-specific thinking — not to test prior knowledge. There are no correct answers; behavioral depth is the signal.

No external installs required. No Weka, no Postman, no downloaded datasets. Every task is self-contained within the platform.

Validation replaces quizzes. Multiple choice questions about accuracy ranges or HTTP methods are trivially guessable and validate nothing. The tasks below are not guessable without engaging with the material.

5.1 Validation Tasks by Thinking Style

The Experience Shaper & The Problem Definer (Frontend / Product)

Task A — Component Highlight + Tag	
Interaction	A before/after of the UI from Activity 2 is shown side by side. The user selects 2 of 5 reasoning tags that best describe what motivated their biggest change: 'Fixed visual hierarchy' / 'Brought error closer to the field' / 'Made the CTA more prominent' / 'Reduced visual clutter' / 'Matched the reading order users expect'.
Signal captured	The combination of tags chosen — not just any individual tag — is the behavioral signal. Choosing 'Reduced visual clutter' AND 'Matched the reading order' signals experience-design awareness. Choosing 'Made the CTA more prominent' alone signals conversion-rate thinking.
AI role	AI scores the tag combination against the user's actual rearrangement to check consistency. Inconsistent tag selections (claiming a motivation that contradicts the actual drag pattern) are flagged as low-confidence.

Task B — User Story Ranking	
Interaction	Five user stories are shown for a fictional feature. The user drags them into a priority order. Stories range from 'As a power user, I want to export all my data as CSV' to 'As a new user, I want to understand what this tool does in 10 seconds'.
Signal captured	Order of stories, speed of decision, and whether they revisit their ranking all feed into the signal. A user who prioritises new-user comprehension over power-user export is showing product instinct.
AI role	AI writes a one-sentence observation about the user's prioritisation pattern for display in the final result — e.g. 'You consistently put new user comprehension above power-user utility, which maps well to early-stage product thinking.'

The Systems Thinker & The Reliability Keeper (Backend / DevOps)

Task A — Live API Response Triage	
Interaction	A real JSON response from a public API (e.g. Open Meteo weather API) is shown in a formatted panel. The user is asked to drag 'useful' and 'redundant' labels onto fields. They then write the endpoint's purpose in their own words — a 30-word text field, the only typing in the entire assessment.
Signal captured	The 30-word field is retained here because it is the only way to validate genuine comprehension of a JSON structure — a user who understood the response will describe it accurately; one who did not cannot fake it in 30 words. The field is not graded for prose quality, only for domain relevance.
AI role	AI reads the 30-word description and scores it on a 3-point scale: correctly identifies the data domain (1pt), mentions the key fields (1pt), identifies at least one redundant or surprising field (1pt). Score feeds into Backend confidence.

Task B — Failure Mode Mapping	
Interaction	A simplified architecture diagram of 5 components is shown (load balancer → app server → database → cache → monitoring). A failure scenario is described: 'The cache layer has gone down at peak traffic.' The user drags consequence tags onto each other component: 'Unaffected' / 'Degraded' / 'Will fail' / 'Will slow significantly'.
Signal captured	Which components the user correctly identifies as degraded vs. failed — and how quickly they trace the cascade — is the signal. A DevOps thinker will reach for monitoring first. A Backend thinker will think about the database fallback path.
AI role	Rule-based scoring: correct cascade identification scores +2 DevOps or +2 Backend depending on investigation order. AI not used for scoring here — the signal is deterministic.

The Signal Seeker & The Hypothesis Tester (Data / AI)

Task A — Anomaly Detection in a Live Dataset	
Interaction	A 20-row table of fictional user activity data is shown with 3 deliberate anomalies embedded (a negative session duration, an impossibly high event count in one minute, a user created 3 years before the platform launched). The user places flag markers on any rows they find suspicious and ranks their top finding by severity.
Signal captured	Which anomalies are found, in what order, and whether the severity ranking is logical are all captured. A user who finds all three and correctly ranks the timeline anomaly as most severe shows strong data instinct. Speed to first flag is also recorded.
AI role	AI writes a personalised observation: e.g. 'You noticed the timestamp inconsistency before the statistical outlier — that sequencing suggests

	you approach data with a story-first rather than distribution-first lens, which is common in data engineering roles.'

Task B — Model Output Interpretation	
Interaction	A simple confusion matrix is shown for a binary classifier (spam / not-spam), with precision, recall, and accuracy pre-calculated and displayed. The user is asked to drag 3 decision tags onto the 'Should we ship this model?' prompt: 'Yes, accuracy is sufficient' / 'No — the false negative rate is too high for this use case' / 'Depends on the cost of each error type' / 'Need more data before deciding' / 'Run A/B test first'.
Signal captured	Tag selection pattern is the signal. A user who picks 'Depends on the cost of each error type' and 'No — false negative rate too high' together is showing model evaluation maturity. A user who picks 'Yes, accuracy is sufficient' without qualification shows a common entry-level misconception.
AI role	AI generates a brief calibration note shown at the result stage: e.g. 'Your instinct to weigh false negative cost separately from overall accuracy is the right frame for production ML decisions.'

The Threat Anticipator & The Trust Architect (Cybersecurity)

Task A — Threat Model Builder	
Interaction	A simplified data flow diagram is shown: User → Browser → API → Database → Admin Panel. Four threat type cards are shown: 'Injection attack', 'Broken access control', 'Data in transit interception', 'Credential stuffing'. The user drags each threat to the point in the flow where it is most likely to occur.
Signal captured	Correct placement, speed, and whether the user hesitates at the Admin Panel (the highest-value target) are all signals. Correctly placing broken access control at the Admin Panel boundary within 10 seconds is a strong Cybersecurity signal.
AI role	Rule-based scoring. No AI required — placements are unambiguous enough to score deterministically.

Task B — Security Review Reaction Card	
Interaction	A 12-line code snippet is shown — a basic authentication function with 2 obvious flaws (plaintext password comparison, no input sanitisation) and 1 subtle flaw (timing attack vulnerability in the comparison logic). The user places 'Flag' markers on lines they consider problematic.
Signal captured	Flagging the timing attack on the comparison logic is the strong signal — this requires knowing that character-by-character string comparison leaks information through response time variation. Users who find only the obvious flaws get a partial signal. Speed to first flag on the subtle issue is the key metric.
AI role	AI writes a personalised note for the result stage based on which flaws were found and in what order.

6. Stage 5 — Scoring Engine

All scoring in Stages 2 and 4 is rule-based, deterministic, and auditable. AI is never used for scoring. This is intentional — users must be able to understand why a domain appeared, and that requires the scoring logic to be traceable.

6.1 Pre-Assessment Scoring (Stage 2)

Signal	Primary domain +pts	Secondary domain +pts	Notes
API + DB prioritised in Act. 1	**Backend +3**	—	*Order matters; DB after API scores higher than DB first*
ML model included in Act. 1	**AI/ML +2**	Data +1	*Only if placed logically between data and response*
UI Response anchored deliberately	**Frontend +2**	Product +1	*Anchor = placed first or last with 0 rearrangement after*
Firewall added first in Act. 4	**Cybersecurity +4**	Backend +1	*Strong signal — most users miss it entirely*
Error message paired to field in Act. 2	**Frontend +2**	Product +2	*Both domains rewarded equally*
Deployment history pinned first in Act. 3	**DevOps +4**	Backend +1	*Highest-value debug signal in this activity*
URL token flagged in Act. 5	**Cybersecurity +4**	—	*Subtle vulnerability — strong discriminator*
In-app notifs → Needs more thought in Act. 6	**Product +3**	—	*Evidence-based caution is the signal*
2FA shipped without prompting in Act. 6	**Cybersecurity +2**	Product +1	*Compliance awareness without user demand*
All 9 architecture components added in Act. 4	**DevOps +1**	Product +1	*Completionism signal — moderate value only*
Reaction card A selected in Act. 2	**Frontend +1**	—	*Holistic visual instinct*
Reaction card B selected in Act. 2	**Product +2**	Frontend +1	*Flow and user journey reasoning*

6.2 Validation Task Scoring (Stage 4)

Validation tasks add a multiplier to existing domain scores — they do not replace them. A domain that scored 8 points in the pre-assessment with a 1.5× validation multiplier outranks a domain that scored 10 points in the pre-assessment with no validation completed.

Validation outcome	Multiplier	Example
Strong engagement — all signals positive	**×1.8**	*Anomaly detection: found all 3, ranked correctly, fast first flag*
Good engagement — majority of signals positive	**×1.4**	*Threat map: 3 of 4 threats placed correctly*
Partial engagement — mixed signals	**×1.1**	*Feature triage: all to Needs More Thought — low conviction*
Low engagement — few signals captured	**×0.9**	*UI task: completed but reaction card skipped*
Task skipped or abandoned	**×0.7**	*User exits before completing either validation task*

7. AI Integration — Where and Why

AI is used in exactly three places. Everywhere else, logic is rule-based, deterministic, and faster. This separation ensures the platform remains trustworthy and explainable — users can always trace why a domain appeared.

7.1 AI Touchpoints

AI Touchpoint 1 — Thinking Style Reveal Copy (Stage 3)

After the rule-based engine selects 3–4 thinking styles, the AI generates the personalised explanation beneath each card. It receives the user's full behavioral log and writes a 2–3 sentence description referencing specific actions the user took.

Example prompt sent to AI:

Thinking style shortlisted: The Threat Anticipator

User behavioral log:
- Activity 4: Firewall added first (t=4s), before load balancer (t=11s)
- Activity 5: URL token flagged at t=7s; DOM password at t=14s; CAPTCHA at t=22s
- Activity 6: 2FA moved to Ship Now; In-app notifications moved to Cut It

Write 2 sentences explaining why this style was matched. Be specific about the user's actions. Do not use generic praise. Do not use the word "instinct".

Example AI output:

"You reached for the firewall before the load balancer — most people building for scale think about traffic first. You flagged the session token in the URL within 7 seconds, before the more obvious DOM vulnerability, which means you were scanning the attack surface rather than the visible interface."

AI Touchpoint 2 — Validation Task Observation Notes (Stage 4)

After specific validation tasks, the AI writes a one-sentence calibration note that is shown to the user at the result stage. This anchors the domain bubble to a real moment in the session.

Used for: Anomaly Detection, Model Output Interpretation, and Security Code Review tasks. Not used for purely deterministic tasks like Failure Mode Mapping or Threat Model Builder.

AI Touchpoint 3 — Learning Roadmap Personalisation (Stage 6)

The standard domain roadmap (4 stages) is re-annotated by AI based on gaps and strengths observed during the assessment. Stages the user demonstrated competency in are marked 'You already showed

this' with evidence. Stages where the user showed uncertainty are marked 'Start here' with a specific first-step recommendation.

Example annotation for a Signal Seeker in Data Engineering who found 2 of 3 anomalies:

- Stage 1 — Programming & statistics: 'You already showed this — your anomaly ranking suggested statistical distribution awareness.'
- Stage 2 — Data cleaning and transformation: 'Start here — you found the outlier but missed the timestamp inconsistency. Schema validation is the gap.'

7.2 AI is NOT used for:

- Activity scoring in Stage 2 — rule-based only
- Thinking style selection in Stage 3 — rule-based only
- Validation task scoring in Stage 4 — rule-based only
- Bubble size calculation in Stage 5 — rule-based only

This separation means the system is always explainable. If a user asks why Backend appeared as their largest bubble, the answer is a specific list of behavioral signals — not 'the AI decided.'

8. Domain Coverage Summary

All seven domains are fully represented across both the pre-assessment and validation stages. The table below maps each domain to its primary activities, thinking styles, and validation tasks.

Domain	Primary activities	Thinking style(s)	Validation tasks	Key discriminator
Backend Engineering	1, 3, 4	*Systems Thinker*	Live API Triage, Failure Mode Mapping	DB + API prioritisation order
Frontend Development	2, 1	*Experience Shaper*	Component Highlight + Tag, User Story Ranking	Error message pairing
DevOps / Infrastructure	3, 4	*Reliability Keeper*	Failure Mode Mapping	Deployment history first pin
Data Engineering	1, 4	*Signal Seeker*	Anomaly Detection, Model Output	Outlier ranking accuracy
AI / Machine Learning	1, 4, 6	*Hypothesis Tester, Signal Seeker*	Anomaly Detection, Model Output	ML model placement + model evaluation maturity
Cybersecurity	4, 5, 6	*Threat Anticipator, Trust Architect*	Threat Model Builder, Security Code Review	URL token flag timing
Product Engineering	2, 6	*Problem Definer, Experience Shaper*	User Story Ranking, Feature Triage	Evidence-first triage pattern

9. Key Design Decisions & Rationale

9.1 Why rule-based scoring, not AI scoring?

Scoring must be auditable. When a user asks 'why did Cybersecurity appear?', the answer needs to be a specific list of actions — not 'the model decided.' This is especially important for a career platform where users are making consequential decisions. AI generates language; logic makes decisions.

9.2 Why no typing except the 30-word API description?

Typing interrupts flow and is easy to fake with generic sentences. The one exception — describing a live API response — is retained because JSON comprehension cannot be faked in 30 words without actually reading the response. All other validation uses drag, flag, rank, and tag interactions that force engagement with the actual content.

9.3 Why are reaction cards 3 options, not free text?

Three carefully crafted options can each be genuinely attractive to a different type of thinker. The constraint forces a real choice that reveals the user's reasoning priority. Free text would produce noise — vague, generic, and impossible to score consistently.

9.4 Why does the scoring engine use multipliers, not additive points?

Additive scoring rewards completion over genuine signal. A user who guesses well across 6 activities might outscore a user who showed exceptional depth in 2. Multipliers let strong validation performance correct and amplify weak pre-assessment signals, and let weak validation performance flag low-confidence shortlisting.

9.5 Why are there 8 thinking styles, not 7 domains?

Domains are where you work. Thinking styles are how you work. Some domains (Data Engineering and AI/ML) have genuinely different thinkers inside them — the Signal Seeker who loves finding patterns in existing data, and the Hypothesis Tester who wants to run experiments and build models. Mapping to styles first lets the platform route users to the right entry point within a domain, not just to the domain name.